

Miranda 実装方針

対象論文

略称	論文
[2020]	Alexandrino, Miranda et al. — <i>Length-weighted λ-rearrangement distance</i> , J. Comb. Optim. 2021
[2021]	Miranda, Alexandrino et al. — <i>Approximation Algorithms for Sorting λ-Permutations by λ-Operations</i> , Algorithms 2021

扱う問題の全体像

両論文は「順列のソート」を扱う。順列でゲノムを表現し、一方を恒等順列 $\iota = (1\ 2\ \dots\ n)$ とすると、もう一方のゲノムをソートする操作の最小コスト列を求める問題になる。

操作の定義

操作	記号	定義	サイズ
Unsigned reversal	$\rho(i, j)$	$\pi_i \dots \pi_j$ を逆順に	$j - i + 1$
Signed reversal	$\bar{\rho}(i, j)$	$\pi_i \dots \pi_j$ を逆順にし符号反転	$j - i + 1$
Transposition	$\tau(i, j, k)$	$\pi_i \dots \pi_{j-1}$ と $\pi_j \dots \pi_{k-1}$ を交換	$k - i$

サイズ $\leq \lambda$ の操作を **λ -操作** と呼ぶ。

[2020] の問題：長さ重み付き λ -整列距離

操作 β のコストを $f(\beta) = |\beta|^\alpha$ と定義 ($\alpha \geq 1$ は実数パラメータ)。

問題名	略称	順列種	操作
Sorting by Length-Weighted λ -Reversals	SbR	unsigned	λ -reversal
Sorting by Length-Weighted λ -Transpositions	SbT	unsigned	λ -transposition
Sorting by Length-Weighted λ -Reversals & Transpositions	SbRT	unsigned	both
Sorting by Length-Weighted Signed λ -Reversals	Sb $\bar{R}$	signed	signed λ -reversal
Sorting by Length-Weighted Signed λ -Reversals & Transpositions	Sb $\bar{R}T$	signed	both

[2021] の問題： λ -順列の λ -操作によるソート

λ -順列：全 i に対して $||\pi_i| - i| < \lambda$ を満たす順列（各要素が正規位置から λ 未満の距離にある）。

操作適用後も λ -順列を保つ制約がある。

問題	略称	近似率
Unsigned λ -perm by λ -reversals	dr λ	$O(\lambda^2)$ または $O(\lambda)$
Unsigned λ -perm by λ -transpositions	dt λ	$O(1)=12$
Unsigned λ -perm by λ -reversals & transpositions	drt λ	$O(1)=12$
Signed λ -perm by λ -reversals	d $\bar{r}\lambda$	$O(\lambda^2)$ または $O(\lambda)$
Signed λ -perm by λ -reversals & transpositions	d $\bar{r}t\lambda$	$O(1)=12$

実装すべきもの

1. 基本操作

既存の `srcs/permutation/` を拡張する。

```
srcs/permutation/
  permutation.c      ← 既存 (identity, compose, breakpoints等)
  generate_permutation.c ← 既存 (next_permutation等)
  breakpoint_graph.c ← 既存 (SBPBI用)
  reversal.c         ← 新規
  transposition.c    ← 新規
  lambda_permutation.c ← 新規 ([2021]専用の条件チェック等)
```

reversal.c

```
// unsigned reversal:  $\pi$  に  $\rho(i,j)$  を適用 (0-indexed)
void apply_reversal(int *perm, int n, int i, int j);

// signed reversal:  $\pi$  に  $\rho(i,j)$  を適用 (符号反転込み)
void apply_signed_reversal(int *perm, int n, int i, int j);

// サイズ取得
int reversal_size(int i, int j); // = j - i + 1
```

transposition.c

```
//  $\pi$  に  $\tau(i,j,k)$  を適用 (0-indexed:  $[i,j)$  と  $[j,k)$  を交換)
void apply_transposition(int *perm, int n, int i, int j, int k);

// サイズ取得
int transposition_size(int i, int k); // = k - i
```

lambda_permutation.c

```
//  $\lambda$ -順列判定: 全  $i$  で  $||\text{perm}[i]| - (i+1)| < \lambda$  (1-indexed値 / 0-indexed位置)
int is_lambda_perm(int *perm, int n, int lambda);
```

```
// λ-順列の生成（ランダム、またはランダムλ-操作をk回適用）
int *gen_lambda_perm_random(int n, int lambda);
int *gen_lambda_perm_from_id(int n, int lambda, int k_ops);
```

2. 解析用計算（ヘルパー）

srcs/permutation/perm_analysis.c として追加（または permutation.c に追記）。

2.1 Inversions（逆転数）

```
// Inv(π) : (πi, πj) s.t. i < j, |πi| > |πj| の個数
int count_inversions(int *perm, int n);

// ΔInv(π, reversal p(i,j)) : 逆転数の変化量（適用前後の差）
int delta_inv_reversal(int *perm, int n, int i, int j);

// ΔInv(π, transposition τ(i,j,k))
int delta_inv_transposition(int *perm, int n, int i, int j, int k);
```

- 実装メモ：ナイーブに $O(n^2)$ 、または逆転数の変化を区間内のみ走査して $O(\lambda^2)$ で十分。
- 高精度版は Fenwick tree で $O(\lambda \log \lambda)$ ([2021] 論文で言及、初期実装は $O(\lambda^2)$ で可)。

2.2 Entropy（エントロピー）

```
// ent(πi) = ||πi| - i| (1-indexed 位置)
int entropy_element(int pi_i, int pos_1indexed);

// ent(π) = Σ ent(πi)
int entropy_perm(int *perm, int n);

// Δent(π, reversal) / Δent(π, transposition)
int delta_ent_reversal(int *perm, int n, int i, int j);
int delta_ent_transposition(int *perm, int n, int i, int j, int k);

// E-even: 負要素かつ entropy 偶数 の個数
// E+odd: 正要素かつ entropy 奇数 の個数
int count_E_minus_even(int *perm, int n);
int count_E_plus_odd(int *perm, int n);
```

2.3 Breakpoints と Strips

```
// unsigned reversal 用 breakpoint: |(πi+1) - πi| != 1
// transposition / signed reversal 用 breakpoint: πi+1 - πi != 1
int count_breakpoints_unsigned_rev(int *perm, int n);
int count_breakpoints_transposition(int *perm, int n);
int count_breakpoints_signed_rev(int *perm, int n);

// strip 列挙: returns strip array (start, end, type=INC/DEC)
typedef struct { int start; int end; int is_increasing; } Strip;
Strip *find_strips(int *perm, int n, int is_signed, int *strip_count);
```

```
// 最小の場外要素の位置を返す
int find_smallest_out_of_place(int *perm, int n);
```

3. コスト関数

[2020] で使うコスト $f(\beta) = |\beta|^\alpha$ 。

```
// cost = size^alpha
double rearrangement_cost(int size, double alpha);

// 操作列の総コスト
double total_cost(int *sizes, int m, double alpha);
```

4. [2020] のアルゴリズム

`srcs/algorithms/` を新設（または `sbpbi/` に倣い `srcs/length_weighted/`）。

Algorithm 1: Inversion-based greedy (unsigned, $\alpha=1$)

```
// SbR, SbT, SbRT 用
// 各ステップで  $\Delta \text{Inv}(\pi, \beta) / f(\beta)$  最大の  $\lambda$ -操作を選択
// mode: MODE_REVERSAL, MODE_TRANSPOSITION, MODE_BOTH
typedef struct { int i; int j; int k; int type; } LWOp;

LWOp *alg1_inversion_greedy(int *perm, int n, int lambda, double alpha,
                           int mode, int *op_count);
```

近似率：

- SbR, SbRT: $(\lambda-1)$ -approx
- SbT: $(\lambda/2)$ -approx
- $\lambda=3$ のとき: SbR, SbRT $\rightarrow$ 2-approx、SbT $\rightarrow$ 4/3-approx

Algorithm 2: Entropy-based greedy (unsigned, $\alpha=1$)

```
// 各ステップで  $\Delta \text{ent}(\pi, \beta) / f(\beta)$  最大の  $\lambda$ -操作を選択
// スコアが 1/2 未満になったら Lemma 9 の super short 列に切り替え
LWOp *alg2_entropy_greedy(int *perm, int n, int lambda, double alpha,
                           int mode, int *op_count);
```

近似率： λ -approx for SbR, SbT, SbRT

Algorithm 3: Score function φ (signed)

```
//  $\varphi(\pi, \beta) = \Delta(2 \cdot \text{Inv} + |E^-_{\text{even}}| + |E^+_{\text{odd}}|) / f(\beta)$ 
// 各ステップで  $\varphi$  最大の signed  $\lambda$ -操作を選択
LWOp *alg3_phi_greedy(int *perm, int n, int lambda, double alpha,
                      int mode, int *op_count); // mode: SbRbar, SbRbarT
```

近似率： λ -approx for $Sb\bar{R}$, $Sb\bar{R}T$

Score function ψ (signed、 $\lambda=3$ 専用)

```
//  $\psi(\pi, \beta) = \Delta(2 \cdot \text{Inv} + \text{codd}(G(\pi))) / f(\beta)$   
// codd: 逆転グラフ  $G(\pi)$  の奇連結成分数  
LWOp *alg_psi_greedy_short(int *perm, int n, int *op_count);
```

近似率： $7/3$ -approx for $Sb\bar{R}T$ ($\lambda=3$)

$\alpha \geq 2$ の 2-approx / 3-approx

Theorem 8 ($\alpha \geq 2$ unsigned $\rightarrow$ 2-approx) と Theorem 11 ($\alpha \geq 2$ signed $\rightarrow$ 3-approx) は Algorithm 1/3 をそのまま使い、パラメータ検証のみ。

$\alpha \geq 3$ の多項式時間厳密解 (Theorem 9, 12, 13)

```
//  $\alpha \geq 3$  で最適解を返す (super short reversal/transposition のみ使用)  
LWOp *alg_exact_alpha3(int *perm, int n, int lambda, double alpha,  
                        int mode, int *op_count);
```

5. [2021] のアルゴリズム

srcs/algorithms/lambda_perm/ 以下 (または srcs/permutation/ 内)。

Algorithm A: Inversions-based (unsigned λ -permutation、 $O(\lambda^2)$ -approx)

```
// 各ステップで Inv 最小化、かつ結果が  $\lambda$ -順列になる  $\lambda$ -操作を選択  
LWOp *lp_inversions_greedy_unsigned(int *perm, int n, int lambda, int *op_count);
```

近似率： $O(\lambda^2)$ for $dr\lambda$, $dt\lambda$, $drt\lambda$

Algorithm B: Score-based (signed λ -permutation、 $O(\lambda^2)$ -approx)

```
//  $\text{score}(\pi, \sigma) = \Delta(\text{Inv} + |E^-_{\text{even}}| + |E^+_{\text{odd}}|)$ 、 $\text{score} \geq 1$  を保証  
LWOp *lp_score_greedy_signed(int *perm, int n, int lambda, int *op_count);
```

近似率： $(\lambda(\lambda-1)/2 + \lambda)$ -approx for $d\bar{r}\lambda$, $d\bar{r}t\lambda$

Algorithm C: Breakpoints-based ($O(\lambda)$ -approx または $O(1)$ -approx)

```
// 最小場外要素を含む strip を正位置へ移動  
//  $\lambda$ -reversals のみ  $\rightarrow (10+2\lambda)$ -approx  
//  $\lambda$ -transpositions のみ  $\rightarrow 12$ -approx  
// both  $\rightarrow 12$ -approx  
LWOp *lp_breakpoints_reversals(int *perm, int n, int lambda, int *op_count);  
LWOp *lp_breakpoints_transpositions(int *perm, int n, int lambda, int *op_count);  
LWOp *lp_breakpoints_both(int *perm, int n, int lambda, int *op_count);
```

内部で使う構成要素（直接呼ぶことは少ない）：

```
// Lemma 7: 増加 strip S を  $\leq 4$  個の  $\lambda$ -transposition で正位置へ
void move_strip_transpositions(int *perm, int n, int lambda,
                               Strip *S, LWOp *out, int *count);

// Lemma 8: 増加 strip S を  $\leq (5+\lambda-1)$  個の  $\lambda$ -reversal で正位置へ
void move_strip_reversals(int *perm, int n, int lambda,
                           Strip *S, LWOp *out, int *count);

// Lemma 9: 減少 strip S を  $\leq 2$  操作 (transposition + reversal) で処理
void move_strip_decreasing(int *perm, int n, int lambda,
                            Strip *S, LWOp *out, int *count);

// Lemma 10: 減少 strip を 1 つの  $\lambda$ -reversal で増加 strip 化 ( $\lambda$ -perm 保持確認)
int reverse_decreasing_strip(int *perm, int n, int lambda, Strip *S);
```

6. 逆転グラフ $G(\pi)$ ([2020] の ψ アルゴリズムに必要)

srcs/graph/inversion_graph.c

```
//  $G(\pi)$  の隣接リスト構築 (頂点:  $|\pi|$ 、辺: inversion)
Vertex **build_inversion_graph(int *perm, int n);

// 連結成分数  $c(\pi)$  と奇連結成分数  $codd(\pi)$  の計算
int count_components(Vertex **g, int n, int *codd_out);
```

7. ヘッダ構成

```
includes/
permutation.h      ← reversal, transposition, lambda_perm の宣言追加
rearrangement.h    ← LWOp 型、コスト関数、アルゴリズムのエントリーポイント
```

rearrangement.h の骨格：

```
#ifndef REARRANGEMENT_H
# define REARRANGEMENT_H

# define MODE_REVERSAL      1
# define MODE_TRANSPOSITION 2
# define MODE_BOTH          3

typedef struct {
    int type; // MODE_REVERSAL or MODE_TRANSPOSITION
    int i, j; // reversal  $\rho(i,j)$  または transposition の  $i,j$ 
    int k;    // transposition  $\tau(i,j,k)$  のみ使用
} LWOp;
```

```

double rearrangement_cost(int size, double alpha);

// [2020] algorithms
LWOp *alg1_inversion_greedy(int *perm, int n, int lambda, double alpha,
                           int mode, int *op_count);
LWOp *alg2_entropy_greedy (int *perm, int n, int lambda, double alpha,
                           int mode, int *op_count);
LWOp *alg3_phi_greedy      (int *perm, int n, int lambda, double alpha,
                           int mode, int *op_count);
LWOp *alg_psi_greedy_short (int *perm, int n, int *op_count);
LWOp *alg_exact_alpha3     (int *perm, int n, int lambda, double alpha,
                           int mode, int *op_count);

// [2021] algorithms
LWOp *lp_inversions_greedy_unsigned(int *perm, int n, int lambda, int *op_count);
LWOp *lp_score_greedy_signed       (int *perm, int n, int lambda, int *op_count);
LWOp *lp_breakpoints_reversals     (int *perm, int n, int lambda, int *op_count);
LWOp *lp_breakpoints_transpositions(int *perm, int n, int lambda, int *op_count);
LWOp *lp_breakpoints_both          (int *perm, int n, int lambda, int *op_count);

#endif

```

8. 順列の表現規約

論文と既存ライブラリで表現が異なるため整理しておく。

項目	論文	本ライブラリ
インデックス	1-based ($\pi_1 \dots \pi_n$)	0-based (perm[0]...perm[n-1])
unsigned 値	1...n	0...n-1 (is_natural=0) または 1...n (is_natural=1)
signed	$\pm 1 \dots \pm n$	正負の int (
恒等順列	(1 2 ... n)	perm[i] = i (0-indexed) または perm[i] = i+1

方針：ライブラリは既存の 0-indexed 規約を維持し、論文の添字変換はアルゴリズム実装内で吸収する。signed の場合は int の正負で符号を表す (-0 は使わない)。

9. 実装優先順位

1. フェーズ 1 — 基本操作 (前提)

- apply_reversal, apply_signed_reversal, apply_transposition
- count_inversions, delta_inv_*
- entropy_*, count_E_minus_even, count_E_plus_odd
- count_breakpoints_*, find_strips, is_lambda_perm

2. フェーズ 2 — [2021] アルゴリズム (実験的に検証しやすい)

- lp_inversions_greedy_unsigned (Theorem 2、 $O(\lambda^2)$ -approx)
- lp_breakpoints_transpositions (Theorem 5、12-approx)
- lp_breakpoints_both (Theorem 6、12-approx)

- `lp_breakpoints_reversals` (Theorem 4、 $O(\lambda)$ -approx)
- `lp_score_greedy_signed` (Theorem 3)

3. フェーズ 3 – [2020] アルゴリズム (コスト関数付き)

- `rearrangement_cost`
- `alg1_inversion_greedy` (Algorithm 1)
- `alg2_entropy_greedy` (Algorithm 2)
- `alg3_phi_greedy` (Algorithm 3)
- `alg_psi_greedy_short` (ψ 、 $\lambda=3$ 特化)
- `alg_exact_alpha3` ($\alpha \geq 3$ 厳密解)

4. フェーズ 4 – ベンチマーク・検証ツール

- ランダム λ -順列生成
- 各アルゴリズムの近似率を実測して理論値と比較するツール

10. 既存コードの再利用

既存関数	再利用箇所
<code>count_breakpoints</code> (<code>permutation.c</code>)	フェーズ 1 の breakpoint 計算の雛形
<code>first_breakpoint_position</code>	strip 検出の出発点
<code>rand_perm</code> (<code>utils_math.c</code>)	ランダム順列生成
<code>next_permutation</code>	全順列探索による小規模テスト
<code>xmalloc</code> / <code>xfree</code>	全アルゴリズムのメモリ管理

参考：近似率サマリー

[2020] Table 1

パラメータ	SbR	SbT	SbRT	Sb$\bar{R}$	Sb$\bar{R}T$
$\lambda > 1, \alpha = 1$	$\lambda - 1$	$\lambda/2$	$\lambda - 1$	λ	λ
$\lambda = \Theta(n), \alpha = 1$	$O(\lg n)$	$O(\lg^2 n)$	$O(\lg^2 n)$	$O(\lg n)$	$O(\lg^2 n)$
$\lambda = 3, \alpha = 1$	2	4/3	2	3	7/3
$\lambda > 1, \alpha \geq 2$	2	2	2	3	3
$\lambda > 1, \alpha \geq 3$	Exact	Exact	Exact	Exact	Exact

[2021] Table 1 & 2

アルゴリズム	drλ (Reversal)	dtλ (Trans)	drtλ (Both)	d$\bar{r}\lambda$ (S.Rev)	d$\bar{r}t\lambda$ (S.Both)
Breakpoints-based	$10 + 2\lambda$	12	12	$10 + 2\lambda$	12

Inversions- based	$O(\lambda^2)$	$O(\lambda^2)$	$O(\lambda^2)$	$\lambda(\lambda-1)/2+\lambda$	$\lambda(\lambda-1)/2+\lambda$
----------------------	----------------	----------------	----------------	--------------------------------	--------------------------------